JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 17 OF 17 - STUDY STEP 17

Dynamic Programming: State, Transitions, and Optimization

Memoization, Tabulation, Reconstruction, Dimensions, and Space Compression

BY

Vinay Reddy Kalluri

Derive dynamic programs from state meaning, transitions, base cases, evaluation order, and reconstruction instead of memorizing tables.

STATE | TRANSITION | MEMOIZATION | TABULATION | RECONSTRUCTION | DP

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `DSA 16 – Greedy Algorithms`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume makes dynamic programming a specification process: define exactly what a state means, how it depends on smaller states, and what information can be compressed.

Readiness map

Decision	Evidence
START HERE	Subproblems overlap; A choice affects a reusable suffix or prefix state; Brute-force recursion repeats the same arguments; The problem asks for optimum, count, or feasibility under constraints.
PREPARE FIRST	Study Steps 01-16; Recursion, arrays, strings, and complexity.
MOVE ON WHEN	Define DP state and transitions precisely; Choose memoization or tabulation; Reconstruct a solution and compress safe dimensions; Analyze state count times transition cost.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 16 - Greedy Algorithms	**YOU ARE HERE** **DSA 17 - Dynamic Programming**	SEGMENT COMPLETE

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Start Here - Your Route Through This Volume 2

Part I - Core Learning 4

Chapter 1 - SDE-2 Dynamic Programming: Derivation, Reconstruction, and Optimization 4

Part II - Practice and Handoff 28

Practice Ladder and Next Steps 28

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - SDE-2 Dynamic Programming: Derivation, Reconstruction, and Optimization

Why dynamic programming feels harder than it is

Dynamic programming (DP) is not a list of formulas. It is a method for organizing repeated subproblems when an optimum or count can be composed from smaller states. Most failures come from choosing an ambiguous state, omitting a base case, evaluating dependencies in the wrong order, or compressing storage before correctness is clear.

At SDE-2 level, derive the recurrence out loud, count states and transitions, identify numeric and memory limits, and reconstruct the requested choices rather than returning only a score. This chapter builds a repeatable derivation across one-dimensional decisions, grids, knapsack, subset and coin problems, subsequences, edit operations, stock state machines, and advanced interval/tree/bitmask families.

Learning objectives

After completing this chapter, you should be able to:

- distinguish overlapping subproblems from greedy-choice structure and brute force;
- derive state, transition, base cases, evaluation order, and answer location;
- move between memoization and tabulation;
- count complexity as number of reachable states times work per state;
- solve house robber, grid path, 0/1 knapsack, subset sum, and coin change;
- derive LIS, LCS, edit distance, and stock transaction states;
- reconstruct selected items or a sequence from a DP table;
- compress memory only when discarded states are no longer dependencies;
- recognize interval, tree, and bitmask DP boundaries; and
- discuss overflow, unreachable sentinels, recursion depth, allocation, and production scale.

The six-step derivation protocol

For every DP problem, write these six lines before code.

1. **State:** what exact subproblem does `dp[...]` answer?
2. **Choice/transition:** which first or last decision relates this state to smaller states?
3. **Base:** what are the smallest valid states and impossible states?
4. **Order:** in which order are all dependencies already known?
5. **Answer:** which state or aggregate is the requested result?
6. **Complexity:** how many states exist and how much work evaluates each?

Example for climbing `n` steps using moves of one or two:

TEXT

```
state: ways(i) = number of ways to reach exactly step i
transition: ways(i) = ways(i - 1) + ways(i - 2)
base: ways(0)=1, ways(negative)=0
order: increasing i, or recursive memoization
answer: ways(n)
states: n + 1, O(1) work each -> O(n) time
```

The empty construction base `ways(0)=1` is important: it provides one way to complete a recurrence exactly, not zero physical movements.

Recognition: DP versus alternatives

DP is promising when:

- a recursive search revisits the same parameter combination;
- the answer for a prefix, suffix, interval, capacity, or finite state can be composed from smaller ones;
- choices interact, so a local greedy ranking lacks an exchange proof;
- the input asks for an optimum, count, feasibility, or reconstruction; and
- the state space is small enough to enumerate.

DP is not automatically appropriate when states are unique, an invariant yields a linear greedy solution, or dimensions make the state space exponential. A memoized exponential state definition is still exponential.

Memoization versus tabulation

Top-down memoization

Write the recurrence as a recursive function and cache results by state. Benefits:

- follows the mathematical definition;
- computes only reachable states; and
- can be easier for irregular transitions.

Costs:

- recursion frames and possible `StackOverflowError`;
- hash/boxing overhead for non-array keys; and
- less predictable locality.

The memo must distinguish "uncomputed" from a legitimate result such as zero. Use a separate seen array, nullable boxed entries, or a sentinel outside the result domain.

Bottom-up tabulation

Allocate states and fill them in dependency order. Benefits:

- no recursion depth risk;
- predictable memory and often better locality; and
- easier rolling-array compression.

Costs:

- may compute unreachable states; and
- order can be less intuitive for interval or tree structures.

The toolkit implements climbing both ways. Both take `O(n)` time. The tabulated version keeps only two predecessors and uses `O(1)` space; memoization uses `O(n)` memo and stack.

Pattern 1: house robber / nonadjacent selection

Given values along a line, select a maximum-sum subset with no adjacent indexes. Define:

```text
dp[i] = best value using prefix [0, i)
```

At last prefix item `i − 1`, either skip it and keep `dp[i − 1]`, or take it and add its value to `dp[i − 2]`:

```text
dp[i] = max(dp[i - 1], dp[i - 2] + value[i - 1])
dp[0] = 0
dp[1] = max(0, value[0])
```

Invariant while scanning: `previousOne` and `previousTwo` are optimal results for the two preceding prefix lengths. Only those states are dependencies, so storage compresses to `O(1)`.

For `[2,7,9,3,1]`, prefix optima are `0,2,7,11,11,12`; choose indexes 0,2,4 for 12. If negative values are allowed and selecting nothing is legal, zero is a valid answer. If at least one item is required, bases and result change.

Time `O(n)`, space `O(1)` for score only. Reconstructing indexes normally needs the full table or stored decisions.

Pattern 2: grid path DP

For minimum cost from top-left to bottom-right with moves only right and down:

```text
dp[row][col] = grid[row][col] + min(dp[row-1][col], dp[row][col-1])
```

The graph of legal moves is a DAG ordered by `row + col`. First cell is its own cost; first row can come only from the left; first column only from above.

For grid:

```text
1 3 1
1 5 1
4 2 1
```

minimum prefix rows become `[1,4,5]`, `[2,7,6]`, `[6,8,7]`, answer 7. One path is right, right, down, down.

Time is `O(rows * cols)`. Score-only storage compresses to one row of `O(cols)`: before updating `dp[col]` it means the value from above; after update it means current row, while `dp[col-1]` is left. Obstacles need an unreachable sentinel and guarded addition. Arbitrary four-direction movement creates cycles and calls for graph shortest-path algorithms, not this acyclic recurrence.

Pattern 3: 0/1 knapsack

Each item has positive weight and nonnegative value and may be taken at most once. With capacity `c`:

```text
dp[i][c] = best value using first i items with capacity c
```

Skip item `i−1`, or take it if its weight fits:

```text
dp[i][c] = max(dp[i−1][c],
               value[i−1] + dp[i−1][c−weight[i−1]])
```

Both branches depend on the previous item row, enforcing at-most-once use. Base row zero is all zero.

TRACE IT | Dry run and reconstruction

Weights `[2,3,4]`, values `[4,5,7]`, capacity 6. Best value is 11 from items 0 and 2. Starting at `dp[3][6]`, compare with `dp[2][6]`; the value differs, so item 2 was selected and capacity becomes 2. Compare `dp[2][2]` with `dp[1][2]`; they match, so skip item 1. Item 0 then differs from the zero row, so select it.

State count is `(n+1)(capacity+1)`, each `O(1)`, yielding `O(nC)` time and space. This is pseudo-polynomial: capacity is numeric magnitude, not input bit length.

The runnable full-table implementation rejects a table above an explicit cell budget and computes dimensions in `long` before any `+1` result reaches an array allocation. In particular, `capacity == Integer.MAX_VALUE` cannot become a negative column count through overflow. A production API should choose its budget from measured heap limits or use score-only compression when reconstruction is unnecessary.

For score only, compress to `O(C)` and iterate capacities downward. Downward order ensures `dp[c−weight]` still represents the previous item row. Iterating upward accidentally permits the same item repeatedly and solves an unbounded variant. Reconstruction becomes harder after compression; retain decisions, rerun portions, or accept full storage.

Pattern 4: subset sum

For nonnegative values, `possible[s]` says whether some processed subset totals `s`. Seed `possible[0]=true`. For each value, iterate `s` downward from target and set:

```text
possible[s] |= possible[s − value]
```

Invariant after an item: `possible[s]` represents subsets using only processed items, each at most once. Descending order prevents reading a state just created by the same item.

Time `O(n * target)`, space `O(target)`. Zero values do not change feasibility but matter for counting subsets. Negative values invalidate the simple `0..target` index range; use an offset range, a set of reachable sums, meet-in-the-middle, or another constraint-specific method.

Equal-partition is subset sum with target `total/2` after checking the total is even. Widen total before addition.

Pattern 5: coin change distinctions

"Coin change" names several different DPs.

Minimum number of coins, unlimited reuse

```text
dp[amount] = minimum coins to form amount
dp[0] = 0
dp[a] = 1 + min(dp[a - coin]) over fitting coins
```

Use an unreachable sentinel and never add one to it blindly. Loop amount increasing so smaller amounts are known. Time `O(target * coinTypes)`, space `O(target)`.

For coins `[1,3,4]`, target 6, minimum is two coins `3+3`; the locally largest coin 4 leaves 2 and produces `4+1+1`, showing why a generic largest-coin greedy rule fails.

Count combinations, unlimited reuse

Seed `ways[0]=1`; loop coins outside and amounts increasing inside. This counts each multiset once. Swapping loop order counts ordered sequences/permutations instead. For 0/1 use, amounts descend. Loop order is part of the problem semantics, not a micro-optimization.

Counts can grow far beyond `long`; define modulus or use `BigInteger` when exact large counts matter.

Pattern 6: longest increasing subsequence

Quadratic DP defines `dp[i]` as LIS length ending exactly at `i`:

```text
dp[i] = 1 + max(dp[j]) for j < i and values[j] < values[i]
```

This takes `O(n^2)` time and `O(n)` space and reconstructs naturally with predecessor indexes.

The `O(n log n)` method maintains `tails[length-1]`, the smallest possible tail value of any increasing subsequence of that length seen so far. For each value, binary-search the first tail greater than or equal to it and replace that tail, or extend the array.

Invariant: tails are sorted, and a smaller tail is at least as extendable as a larger tail for the same length. Replacing a tail does not claim the tails array itself is a subsequence; it preserves existence of some subsequence for each length.

For `[10,9,2,5,3,7,101,18]`, tails evolve `[10]`, `[9]`, `[2]`, `[2,5]`, `[2,3]`, `[2,3,7]`, `[2,3,7,101]`, `[2,3,7,18]`; length 4. Strict LIS uses lower bound `>=`; nondecreasing subsequence uses upper bound `>`.

Reconstructing the `O(n log n)` sequence requires predecessor and tail-index arrays, not just tail values.

Pattern 7: longest common subsequence

For code-point arrays `a` and `b`:

```text
dp[i][j] = LCS length of prefixes a[0..i) and b[0..j)
```

If last code points match, include them: `1 + dp[i−1][j−1]`. Otherwise skip one side: `max(dp[i−1][j], dp[i][j−1])`. Empty prefix rows/columns are zero.

For `ABCBDAB` and `BDCABA`, an LCS length is 4, such as `BCBA`. Reconstruction walks backward: matching symbols are chosen; otherwise move toward a neighboring state with the same optimal value. Ties permit multiple correct sequences, so deterministic tie policy matters.

Time and full reconstruction space are `O(nm)`. Score only compresses to `O(min(n,m))`, but ordinary backtracking information is lost. Hirschberg's algorithm reconstructs with linear additional space through divide and conquer when needed.

Pattern 8: edit distance

Levenshtein distance permits insertion, deletion, and substitution at unit cost.

```text
dp[i][j] = minimum edits from first i code points to first j
base: dp[i][0]=i, dp[0][j]=j
if equal: dp[i][j]=dp[i−1][j−1]
else: 1 + min(delete dp[i−1][j],
              insert dp[i][j−1],
              replace dp[i−1][j−1])
```

Every transition identifies the last operation, so optimal substructure follows by removing that operation. State count `O(nm)`, transition work constant. The toolkit compresses to two rows. To reconstruct the edit script, retain the table or decisions.

Costs can be weighted, transposition can be added under a different recurrence, and Unicode normalization/unit choices belong to the contract. Code-point distance is not grapheme or linguistic distance.

Pattern 9: stock trading as a state machine

For at most `k` transactions, define:

- `cash[t]`: best profit after at most `t` completed sales while holding no stock;
- `hold[t]`: best profit after buying for transaction `t` and currently holding one stock.

For each price:

```text
hold[t] = max(hold[t], cash[t-1] - price)
cash[t] = max(cash[t], hold[t] + price)
```

Impossible hold states start at negative infinity. The implementation uses a safe sentinel so addition cannot overflow. State count is `O(k)` per day; time `O(nk)`, space `O(k)`.

When `k >= n/2`, the transaction cap cannot bind because each profitable transaction needs at least a buy and later sell day. Sum every positive adjacent increase in `O(n)` time.

Cooldown, fees, multiple simultaneous holdings, and mandatory transactions change state and transitions. Write the state meaning first instead of patching conditionals onto a remembered formula.

Advanced pattern boundaries

Interval DP

State spans a contiguous interval, often `[left,right)`. Choose a split or last action inside it. Matrix-chain multiplication, optimal parenthesization, and burst balloons are examples. Fill shorter intervals before longer ones. Typical state count is `O(n^2)` and trying every split yields `O(n^3)` time.

Tree DP

Root the tree and compute states after children. House robber on a tree returns two values per node: best if this node is excluded and best if included. Included forces children excluded; excluded may choose each child's better state. Complexity is linear, but recursion depth may require an iterative postorder.

Bitmask DP

For small `n`, a mask represents a chosen subset. Traveling-salesperson-style state `dp[mask][last]` stores best cost to visit exactly `mask` and end at `last`. There are `n * 2^n` states and up to `n` transitions each, `O(n^2 2^n)` time. This is powerful around `n` in the teens or low twenties, not a polynomial escape from large input.

Digit DP

Count numbers satisfying a property with state such as `(position, tight, started, summary)`. Memoization is valid only when the state includes every future-relevant constraint. This is a recognition boundary rather than a default interview tool.

Reconstruction and compression tension

A score is not always the requested answer. To reconstruct:

- store a parent/choice per state;
- compare a full state with the transition that could have produced it;
- retain predecessor indexes; or
- rerun selected subproblems when memory is more valuable than time.

Compression is safe only when future values no longer need discarded states. Iteration direction matters in knapsack. In-place grid rows rely on old-above/new-left meanings. Reconstruction may require the information compression removes. State this trade-off before claiming `O(1)` space for a problem that must return a path or chosen set.

Runnable Java 21 reference implementation

Run with `java -ea DynamicProgrammingPatterns`.

JAVA (continued 1/11)

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public final class DynamicProgrammingPatterns {
    private static final long NEGATIVE_INFINITY = Long.MIN_VALUE / 4;
    private static final long MAX_KNAPSACK_CELLS = 20_000_000L;

    private DynamicProgrammingPatterns() {
    }

    public record KnapsackResult(long value, List<Integer> selectedIndexes) {
    }

    public static long climbWaysMemoized(int steps) {
        validateStepCount(steps);
        long[] memo = new long[steps + 1];
        Arrays.fill(memo, -1);
        return climb(steps, memo);
    }

    public static long climbWaysTabulated(int steps) {
        validateStepCount(steps);
        if (steps == 0) {
            return 1;
        }
        long twoBack = 1;
        long oneBack = 1;
        for (int step = 2; step <= steps; step++) {
            long current = Math.addExact(oneBack, twoBack);
            twoBack = oneBack;
            oneBack = current;
        }
```

JAVA (continued 2/11)

```
        return oneBack;
    }

    public static long maxNonAdjacentSum(int[] values) {
        requireArray(values);
        long twoBack = 0;
        long oneBack = 0;
        for (int value : values) {
            long current = Math.max(oneBack, Math.addExact(twoBack, value));
            twoBack = oneBack;
            oneBack = current;
        }
        return oneBack;
    }

    public static long minimumGridPath(int[][] grid) {
        int cols = validateGrid(grid);
        long[] best = new long[cols];
        for (int row = 0; row < grid.length; row++) {
            for (int col = 0; col < cols; col++) {
                if (row == 0 && col == 0) {
                    best[col] = grid[row][col];
                } else if (row == 0) {
                    best[col] = Math.addExact(best[col - 1], grid[row][col]);
                } else if (col == 0) {
                    best[col] = Math.addExact(best[col], grid[row][col]);
                } else {
                    best[col] = Math.addExact(Math.min(best[col], best[col - 1]),
                            grid[row][col]);
                }
            }
        }
        return best[cols - 1];
    }
```

JAVA (continued 3/11)

```
    public static KnapsackResult knapsack01(int[] weights, int[] values, int capacity) {
        if (weights == null || values == null || weights.length != values.length
                || capacity < 0) {
            throw new IllegalArgumentException("invalid knapsack input");
        }
        int n = weights.length;
        long rows = (long) n + 1;
        long columns = (long) capacity + 1;
        long cells = Math.multiplyExact(rows, columns);
        if (rows > Integer.MAX_VALUE || columns > Integer.MAX_VALUE
                || cells > MAX_KNAPSACK_CELLS) {
            throw new IllegalArgumentException("knapsack table exceeds allocation budget");
        }
        long[][] best = new long[n + 1][capacity + 1];
        for (int i = 1; i <= n; i++) {
            if (weights[i - 1] <= 0 || values[i - 1] < 0) {
                throw new IllegalArgumentException("positive weights and nonnegative values required");
            }
            for (int remaining = 0; remaining <= capacity; remaining++) {
                best[i][remaining] = best[i - 1][remaining];
                if (weights[i - 1] <= remaining) {
                    best[i][remaining] = Math.max(best[i][remaining],
                            Math.addExact(values[i - 1],
                                    best[i - 1][remaining - weights[i - 1]]));
                }
            }
        }
        List<Integer> selected = new ArrayList<>();
        int remaining = capacity;
        for (int i = n; i >= 1; i--) {
            if (best[i][remaining] != best[i - 1][remaining]) {
                selected.add(i - 1);
                remaining -= weights[i - 1];
```

JAVA (continued 4/11)

```
            }
        }
        Collections.reverse(selected);
        return new KnapsackResult(best[n][capacity], List.copyOf(selected));
    }

    public static boolean subsetSum(int[] values, int target) {
        requireArray(values);
        if (target < 0 || target == Integer.MAX_VALUE) {
            throw new IllegalArgumentException("target must be in 0..Integer.MAX_VALUE-1");
        }
        boolean[] possible = new boolean[target + 1];
        possible[0] = true;
        for (int value : values) {
            if (value < 0) {
                throw new IllegalArgumentException("values must be nonnegative");
            }
            for (int sum = target; sum >= value; sum--) {
                possible[sum] |= possible[sum - value];
            }
        }
        return possible[target];
    }

    public static int minimumCoins(int[] coins, int target) {
        requireArray(coins);
        if (target < 0 || target == Integer.MAX_VALUE) {
            throw new IllegalArgumentException("invalid target");
        }
        for (int coin : coins) {
            if (coin <= 0) {
                throw new IllegalArgumentException("coins must be positive");
            }
```

JAVA (continued 5/11)

```
        }
        int unreachable = target + 1;
        int[] best = new int[target + 1];
        Arrays.fill(best, unreachable);
        best[0] = 0;
        for (int amount = 1; amount <= target; amount++) {
            for (int coin : coins) {
                if (coin <= amount && best[amount - coin] != unreachable) {
                    best[amount] = Math.min(best[amount], best[amount - coin] + 1);
                }
            }
        }
        return best[target] == unreachable ? -1 : best[target];
    }

    public static int lisLength(int[] values) {
        requireArray(values);
        int[] tails = new int[values.length];
        int size = 0;
        for (int value : values) {
            int low = 0;
            int high = size;
            while (low < high) {
                int middle = low + (high - low) / 2;
                if (tails[middle] < value) {
                    low = middle + 1;
                } else {
                    high = middle;
                }
            }
            tails[low] = value;
            if (low == size) {
                size++;
```

JAVA (continued 6/11)

```
            }
        }
        return size;
    }

    public static String longestCommonSubsequence(String first, String second) {
        requireString(first);
        requireString(second);
        int[] a = first.codePoints().toArray();
        int[] b = second.codePoints().toArray();
        int[][] length = new int[a.length + 1][b.length + 1];
        for (int i = 1; i <= a.length; i++) {
            for (int j = 1; j <= b.length; j++) {
                if (a[i - 1] == b[j - 1]) {
                    length[i][j] = length[i - 1][j - 1] + 1;
                } else {
                    length[i][j] = Math.max(length[i - 1][j], length[i][j - 1]);
                }
            }
        }
        List<Integer> reversed = new ArrayList<>();
        int i = a.length;
        int j = b.length;
        while (i > 0 && j > 0) {
            if (a[i - 1] == b[j - 1]) {
                reversed.add(a[i - 1]);
                i--;
                j--;
            } else if (length[i - 1][j] >= length[i][j - 1]) {
                i--;
            } else {
                j--;
            }
```

JAVA (continued 7/11)

```
        }
        Collections.reverse(reversed);
        StringBuilder result = new StringBuilder();
        for (int point : reversed) {
            result.appendCodePoint(point);
        }
        return result.toString();
    }

    public static int editDistance(String first, String second) {
        requireString(first);
        requireString(second);
        int[] a = first.codePoints().toArray();
        int[] b = second.codePoints().toArray();
        if (b.length > a.length) {
            int[] temporary = a;
            a = b;
            b = temporary;
        }
        int[] previous = new int[b.length + 1];
        int[] current = new int[b.length + 1];
        for (int j = 0; j <= b.length; j++) {
            previous[j] = j;
        }
        for (int i = 1; i <= a.length; i++) {
            current[0] = i;
            for (int j = 1; j <= b.length; j++) {
                if (a[i - 1] == b[j - 1]) {
                    current[j] = previous[j - 1];
                } else {
                    current[j] = 1 + Math.min(previous[j - 1],
                            Math.min(previous[j], current[j - 1]));
                }
```

JAVA (continued 8/11)

```
            }
            int[] temporary = previous;
            previous = current;
            current = temporary;
        }
        return previous[b.length];
    }

    public static long maxStockProfit(int[] prices, int transactions) {
        requireArray(prices);
        if (transactions < 0) {
            throw new IllegalArgumentException("transactions must be nonnegative");
        }
        for (int price : prices) {
            if (price < 0) {
                throw new IllegalArgumentException("prices must be nonnegative");
            }
        }
        if (transactions == 0 || prices.length < 2) {
            return 0;
        }
        if (transactions >= prices.length / 2) {
            long profit = 0;
            for (int day = 1; day < prices.length; day++) {
                if (prices[day] > prices[day - 1]) {
                    profit += (long) prices[day] - prices[day - 1];
                }
            }
            return profit;
        }
        long[] cash = new long[transactions + 1];
        long[] hold = new long[transactions + 1];
        Arrays.fill(hold, NEGATIVE_INFINITY);
```

JAVA (continued 9/11)

```
        for (int price : prices) {
            for (int completed = 1; completed <= transactions; completed++) {
                hold[completed] = Math.max(hold[completed], cash[completed - 1] - price);
                cash[completed] = Math.max(cash[completed], hold[completed] + price);
            }
        }
        return cash[transactions];
    }

    private static long climb(int steps, long[] memo) {
        if (steps <= 1) {
            return 1;
        }
        if (memo[steps] != -1) {
            return memo[steps];
        }
        memo[steps] = Math.addExact(climb(steps - 1, memo), climb(steps - 2, memo));
        return memo[steps];
    }

    private static void validateStepCount(int steps) {
        if (steps < 0 || steps > 91) {
            throw new IllegalArgumentException("steps must be in 0..91 for long output");
        }
    }

    private static int validateGrid(int[][] grid) {
        if (grid == null || grid.length == 0 || grid[0] == null
                || grid[0].length == 0) {
            throw new IllegalArgumentException("nonempty grid required");
        }
        int cols = grid[0].length;
        for (int[] row : grid) {
```

JAVA (continued 10/11)

```
            if (row == null || row.length != cols) {
                throw new IllegalArgumentException("grid must be rectangular");
            }
        }
        return cols;
    }

    private static void requireArray(int[] values) {
        if (values == null) {
            throw new IllegalArgumentException("array must not be null");
        }
    }

    private static void requireString(String value) {
        if (value == null) {
            throw new IllegalArgumentException("string must not be null");
        }
    }

    public static void main(String[] args) {
        assert climbWaysMemoized(10) == 89;
        assert climbWaysTabulated(10) == 89;
        assert maxNonAdjacentSum(new int[] {2, 7, 9, 3, 1}) == 12;
        assert maxNonAdjacentSum(new int[] {-4, -2}) == 0;
        assert minimumGridPath(new int[][] {{1, 3, 1}, {1, 5, 1}, {4, 2, 1}}) == 7;

        KnapsackResult packed = knapsack01(new int[] {2, 3, 4},
                new int[] {4, 5, 7}, 6);
        assert packed.value() == 11;
        assert packed.selectedIndexes().equals(List.of(0, 2));
        boolean hugeCapacityRejected = false;
        try {
            knapsack01(new int[0], new int[0], Integer.MAX_VALUE);
```

JAVA (continued 11/11)

```
        } catch (IllegalArgumentException expected) {
            hugeCapacityRejected = true;
        }
        assert hugeCapacityRejected;
        assert subsetSum(new int[] {3, 34, 4, 12, 5, 2}, 9);
        assert !subsetSum(new int[] {3, 34, 4, 12, 5, 2}, 30);
        assert minimumCoins(new int[] {1, 3, 4}, 6) == 2;
        boolean invalidCoinRejected = false;
        try {
            minimumCoins(new int[] {0}, 0);
        } catch (IllegalArgumentException expected) {
            invalidCoinRejected = true;
        }
        assert invalidCoinRejected;

        assert lisLength(new int[] {10, 9, 2, 5, 3, 7, 101, 18}) == 4;
        String lcs = longestCommonSubsequence("ABCBDAB", "BDCABA");
        assert lcs.length() == 4;
        assert isSubsequence(lcs, "ABCBDAB") && isSubsequence(lcs, "BDCABA");
        assert editDistance("kitten", "sitting") == 3;
        assert maxStockProfit(new int[] {3, 2, 6, 5, 0, 3}, 2) == 7;
    }

    private static boolean isSubsequence(String candidate, String source) {
        int index = 0;
        for (int point : source.codePoints().toArray()) {
            if (index < candidate.length() && candidate.codePointAt(index) == point) {
                index += Character.charCount(point);
            }
        }
        return index == candidate.length();
    }
}
```

Complexity and state table

Family	States	Work/state	Time	Compressed space
climb/house robber	`O(n)`	`O(1)`	`O(n)`	`O(1)`
grid right/down	`O(rows*cols)`	`O(1)`	`O(rows*cols)`	`O(cols)`
0/1 knapsack	`O(nC)`	`O(1)`	`O(nC)`	`O(C)` score only
subset sum	`O(nT)` conceptual	`O(1)`	`O(nT)`	`O(T)`
minimum coin change	`O(T)`	`O(coins)`	`O(T*coins)`	`O(T)`
LIS quadratic	`O(n)` states	`O(n)`	`O(n^2)`	`O(n)`
LIS tails	`O(n)`	`O(log n)` search	`O(n log n)`	`O(n)`
LCS/edit distance	`O(nm)`	`O(1)`	`O(nm)`	`O(min(n,m))` score
stock at most k	`O(nk)`	`O(1)`	`O(nk)`	`O(k)`
interval split DP	`O(n^2)`	often `O(n)`	often `O(n^3)`	problem-specific
bitmask last-state	`O(n2^n)`	up to `O(n)`	`O(n^2 2^n)`	exponential

WATCH OUT | Edge cases and common mistakes

1. **State lacks future information.** If two histories with the same key permit different futures, the state is incomplete.
2. **Legitimate zero used as uncomputed.** Separate memo status from values.
3. **Base for empty construction wrong.** Counts often use one empty way; maxima may use zero or negative infinity.
4. **Impossible state added blindly.** Guard sentinels before arithmetic.
5. **Iteration direction changes item reuse.** Descending capacity is 0/1; ascending enables unbounded reuse.
6. **Combination and sequence counts confused.** Coin/amount loop order changes meaning.
7. **All-negative selection contract.** Decide whether choosing nothing is legal.
8. **Grid recurrence used on cyclic moves.** Use graph algorithms when dependencies are not acyclic.
9. **LIS strictness wrong.** Lower-bound versus upper-bound determines duplicate handling.
10. **Compressed score claimed to reconstruct choices.** Preserve decisions or explain recomputation.
11. **Pseudo-polynomial called polynomial in input length.** Capacity/target magnitude matters.
12. **Count overflow.** Combinatorial counts exceed `long` quickly.
13. **Recursive memo stack ignored.** State count does not bound call depth safely for Java.
14. **Huge table allocated from untrusted input.** Validate product before allocation.
15. **Text unit unspecified.** Code points, UTF-16 units, and graphemes produce different sequence states.

SDE-2 production follow-ups

- **Allocation guards:** compute table dimensions in `long`, enforce budgets, and fail before allocating. `O(nm)` may be impossible even when time sounds acceptable.
- **Sparse states:** memo maps can save memory when few states are reachable, but hashing and boxing add cost and cardinality still needs a bound.
- **Rolling storage:** reuse primitive arrays for locality and less GC pressure; clear only required ranges.
- **Overflow contract:** use exact arithmetic, modular counts, saturation, `BigInteger`, or explicit rejection. Silent wraparound destroys optimal comparisons.
- **Reconstruction audit:** return named choices and validate their score independently. This catches tie/backtracking bugs.
- **Incremental changes:** DP tables generally become stale when inputs change. Determine whether localized recomputation is valid or rebuild from a versioned snapshot.
- **Parallelism:** wavefront/grid phases or independent states may parallelize, but dependencies, synchronization, and memory bandwidth can dominate.
- **Approximation:** capacity or state explosion may require scaling, pruning, beam search, or approximation schemes. Label loss of exactness.
- **Persistence:** checkpointing large tables can cost more than recomputation; measure serialization and version recurrence semantics.
- **Observability:** record state count, reachable-state ratio, table bytes, cache hit rate, and reconstruction validation, not raw sensitive inputs.

TRY IT | Exercises with model checkpoints

Exercise 1: reconstruct house robber indexes

Return one optimal nonadjacent index set.

Model checkpoints: retain full prefix scores or decisions; backtrack by comparing skip with take; define ties deterministically; verify no adjacent indexes and recompute sum.

Exercise 2: grid with obstacles and path

Return minimum cost and coordinates.

Model checkpoints: unreachable sentinel; never add to sentinel; store parent direction or compare table predecessors; start/end blocked behavior; `O(rows*cols)` output-aware memory.

Exercise 3: count coin combinations

Count ways to form target with unlimited coins.

Model checkpoints: validate distinct positive denominations or define duplicate treatment; coins outer, amount ascending; `ways[0]=1`; choose `BigInteger` or modulus; contrast permutation loop order.

Exercise 4: edit script

Return insert/delete/replace operations, not only distance.

Model checkpoints: full table or divide-and-conquer reconstruction; define tie order; indexes shift as edits apply, so represent operations against prefixes or apply from a safe direction; verify script transforms input.

Exercise 5: LIS reconstruction

Extend the `O(n log n)` method to return indexes.

Model checkpoints: `tailIndex[length]` stores input index; predecessor of current is previous length's tail index; final chain backtracks; tails values alone are not a sequence; strict duplicate policy.

Exercise 6: tree independent set

Maximum-weight set of nonadjacent tree vertices.

Model checkpoints: per node return `(excluded,included)`; included adds excluded children, excluded adds max child states; postorder; parent prevents traversal back edge; iterative plan for deep trees.

Exercise 7: traveling salesperson bitmask DP

Find minimum Hamiltonian tour for small `n`.

Model checkpoints: state `(mask,last)` includes start; transition from previous endpoint; unreachable sentinel; close tour to start; `O(n^2 2^n)` time and `O(n2^n)` space; reject large n before shifting/allocating.

Interview answer checklist

- **[ ]** I wrote a one-sentence state meaning with exact indexes/ranges.
- **[ ]** I derived every transition from a last or first choice.
- **[ ]** I covered empty, impossible, and smallest bases.
- **[ ]** My evaluation order satisfies every dependency.
- **[ ]** I identified the exact answer state.
- **[ ]** I counted states times transition work.
- **[ ]** I distinguished pseudo-polynomial and exponential complexity.
- **[ ]** I compressed only after proving which history is unnecessary.
- **[ ]** I preserved enough information for requested reconstruction.
- **[ ]** I defined overflow, text units, and allocation limits.
- **[ ]** I can compare memoization, tabulation, greedy, graph, and search alternatives.

RECAP | Summary

Dynamic programming becomes systematic when every solution follows state, transition, base, order, answer, and complexity. Memoization mirrors recursion; tabulation exposes dependency order and compression. House robber retains two prefix states; grid paths follow a DAG; 0/1 knapsack and subset sum depend on descending capacity; coin loop order encodes reuse and ordering semantics. LIS has quadratic and tails-based formulations. LCS and edit distance align two sequences, while stock trading is a finite state machine over days and transaction counts. Interval, tree, and bitmask DP extend the same method. The SDE-2 distinction is honest state counting, reconstructable outputs, safe sentinels and arithmetic, and memory-aware production boundaries.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: Fibonacci, stairs, and one-dimensional choices
- Interview Core: grids, knapsack, subsequences, and string DP
- SDE-2 Follow-up: reconstruct decisions and control memory
- Challenge: interval, tree, or bitmask state

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: DSA 16 - Greedy Algorithms: Recognition, Proofs, and Scheduling

Series index

Return to the complete series index

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, or System Design and Backend first, then follow the books inside that segment in order. Stable study-step codes remain on filenames and links; the clearer segment book codes appear on the website and every PDF cover.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Advanced Java B: Language, OOP, and Modern Java
Java	**JAVA 05**	Advanced Java C: Collections, Streams, and I/O
Java	**JAVA 06**	Advanced Java A: JVM and Execution
Java	**JAVA 07**	Advanced Java D: Concurrency and the Memory Model
Java	**JAVA 08**	Advanced Java E: Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Advanced Java G: Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
System Design	**SD 01**	Advanced Java F: Design, Backend, Testing, and Security
System Design	**SD 02**	MySQL for Java Backend Interviews
System Design	**SD 03**	Hibernate and JPA for Java Backend Interviews
System Design	**SD 04**	Spring Framework for Java Engineers
System Design	**SD 05**	Spring Boot for Java Backend Engineers

SEGMENT	BOOK	FOCUSED LEARNING PATH
System Design	**SD 06**	Spring Data for Java Backend Engineers
System Design	**SD 07**	MongoDB for Java Backend Interviews
System Design	**SD 08**	Redis for Java Backend Interviews
System Design	**SD 09**	Apache Kafka and Spring Kafka
System Design	**SD 10**	Spring Ecosystem Extensions
System Design	**SD 11**	Spring AI for Java Engineers
System Design	**SD 12**	Advanced Java H: Spring Boot and REST APIs
System Design	**SD 13**	Advanced Java I: Persistence, SQL, JPA, and Caching
System Design	**SD 14**	Advanced Java J: Distributed Systems and System Design

Roadmap editions identify planned books whose chapter sets will be expanded one at a time. Existing deep-dive books remain available later in each segment, so no published material is displaced.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 17 of 17 - Study Step 17. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.